SIMATS ENGINEERING

CSA02 - C Programming

ASSIGNMENT 2

CO2 | 50 marks

Operators, Decisions, Loops and Measured Redesign

Predictions, implementation, operation counts and analysis

Complete solutions for Q1-Q7

Student Name: PROBIR SAHA SHOHOM

Register Number: 1911260319

GitHub Repository:
Probir127/CSA02-1911260319-Assignment2

Student and scope

Student: PROBIR SAHA SHOHOM. Register number: 1911260319. Course: CSA02 - C Programming. Total marks: 50.

Repository: https://github.com/Probir127/CSA02-1911260319-Assignment2

This report covers Q1-Q7 with C source, completed supplied programs, recorded predictions, measured operation counts, tests, alternatives and complexity analysis. The source document's top metadata says CO1, while its assignment title and coverage explicitly say Assignment 2 - CO2; this report follows the title and CO2 content.

The executed compiler is Apple Clang 21.0.0 invoked as gcc on arm64 macOS, with -std=c17 -Wall -Wextra -Wpedantic -O2. compiler.txt records the exact version. The repository uses C source and a Bash build script. Inputs are deterministic fixtures embedded in the C programs and listed in each task's inputs.txt. Output PNGs are labelled renderings of captured program output, not screenshots of a personal terminal session. Capture the terminal screenshots required by the brief when personally executing the documented commands. Faculty sharing/course submission is separate.

PREDICTIONS.md was recorded before execution. All supplied completed-code program bodies outside their TODO areas are retained; separate files demonstrate intentional bugs or alternatives.

Q1 - Precedence, associativity and the discount bug (6 marks)

Assumptions and predictions

The given expression has no declared types. For a reproducible integer-division demonstration, price, quantity, senior, total and net are int; price is in whole rupees and senior is exactly 0 or 1. The fixtures are deliberately small enough to avoid int overflow. Monetary policy for the corrected program: apply 15% discount to the gross amount for seniors, apply 5% GST to the discounted amount, round each discount/tax component half-up to paise, then subtract Rs 50 only if the after-tax total exceeds Rs 1000. The brief does not specify whether the rebate changes the GST base; this solution explicitly uses the stated after-tax rebate order.

Predictions were written in PREDICTIONS.md before execution and committed before the output files. The predicted original results, then observed results, are:

Price / quantity / senior	Predicted total / net (Rs)	Observed total / net (Rs)	Correct payable (Rs)
100 / 10 / 1	900 / 900	900 / 900	892.50
999 / 1 / 0	1048 / 998	1048 / 998	998.95
199 / 3 / 1	537 / 537	537 / 537	532.82

How C groups the expression

Multiplication and division have higher precedence than addition/subtraction. Multiplicative operators associate left-to-right, so price*qty*15/100*senior groups as ((((price*qty)*15)/100)*senior). Addition/subtraction also associate left-to-right. Assignment stores the completed right-hand result. These rules define grouping, not a general left-to-right order for evaluating all operands. There are no operand side effects in this total expression, so unspecified operand evaluation order does not change its mathematical result.

For 100 x 10, gross=1000, discount=150 and the original tax=50, yielding 900. Intended GST on 850 is 42.50, so the corrected bill is 892.50. For 999 x 1, 4995/100 truncates to 49 whole rupees; original total=1048, so net=998. Correct GST is 49.95. For 199 x 3, 8955/100 truncates to 89 and 2985/100 truncates to 29, yielding 597-89+29=537. In the corrected calculation discount=89.55, taxable=507.45, GST=25.3725 rounded to 25.37, total=532.82.

The original GST term taxes gross rather than discounted value for seniors. Integer division discards fractional rupees even when senior is false. Merely adding brackets cannot repair a wrong tax base or integer truncation. With floating operands the truncation changes, but the gross-tax logic bug remains. total>1000 is evaluated before the conditional ?: chooses exactly one branch; the strict > rule matches 'above Rs 1000'. The net expression inherits errors in total rather than being independently wrong for the chosen policy. Tests also check exactly 100000 paise and one paisa above it.

Undefined counter statement and compiler evidence

count=count++ + 1 modifies count through both the postfix increment and assignment without the required sequencing between those side effects in C. It has undefined behaviour; no particular observed count is a valid portable result. The file is compiled to an object solely to capture the warning and is never executed. The actual Apple Clang diagnostic from gcc -Wall -Wextra is:

```
Task01/undefined_example.c:2:42: warning: multiple unsequenced modifications to 'count'
    [-Wunsequenced]
    2 | int main(void) { int count=0; count=count++ + 1; return count; }
      |                                     ~     ^
1 warning generated.
```

The correction is ++count or count += 1 in its own statement. GNU GCC documents the related -Wsequence-point warning under -Wall; the local compiler reports -Wunsequenced. A warning diagnoses the problem rather than defining a result.

Corrected design and alternative complexity

Use both intermediate variables and parentheses to make policy order visible:

```
gross = (long long)price * 100 * qty;
discount = senior ? (gross * 15 + 50) / 100 : 0;
taxable = gross - discount;
tax = (taxable * 5 + 50) / 100;
after_tax = taxable + tax;
payable = after_tax > 100000 ? after_tax - 5000 : after_tax;
```

These formulas apply to non-negative, bounded paise. A real input-facing application must check price/quantity and product limits before multiplication. This demonstration uses fixed, asserted fixtures and does not claim to accept arbitrary input.

The original source repeats price*qty three times. A compiler may eliminate that repeated multiplication. Intermediate variables compute gross once and preserve component amounts for auditing; both designs remain O(1) time and O(1) space per bill. A handful of long long temporaries may occupy registers or stack slots, and optimization can remove them. The corrected algorithm's rounding and tax base intentionally change its arithmetic; its clarity is the main reason for choosing it, not an unsupported speed claim.

```
price,qty,senior,predicted_total,observed_total,predicted_net,observed_net,correct_paise
100,10,1,900,900,900,900,89250
gross=100000 discount=15000 taxable=85000 GST=4250 payable=89250 paise
999,1,0,1048,1048,998,998,99895
gross=99900 discount=0 taxable=99900 GST=4995 payable=99895 paise
199,3,1,537,537,537,537,53282
gross=59700 discount=8955 taxable=50745 GST=2537 payable=53282 paise
Correct counter update: ++count; count=1
Rebate threshold: 100000 paise unchanged; 100001 paise -> 95001
```

Q2 - Short-circuit evaluation (4 marks)

Experiment and predictions

Concession means (senior AND eligible class) OR verified disability card. A deterministic stub models the expensive card check by incrementing calls; it returns true when passenger index is divisible by 10. For indices 0..999, senior is index%4==0 and eligible is index%3!=0. No actual server or passenger records are used.

There are 250 seniors, of whom 84 have indices divisible by 12 and are ineligible, leaving 166 eligible seniors. Checking this cheap branch first makes 1000-166=834 card calls. There are 100 card holders and 33 overlap with eligible seniors,

so 166+100-33=233 concessions. These are the predictions, independently checked by assertions.

```
0 && verify: calls=0
1 || verify: calls=0
0 & verify: calls=1
version,passengers,card_calls,grants
eligible_first,1000,834,233
card_first,1000,1000,233
nested_if,1000,834,233
bitwise,1000,1000,233
audit_all,1000,1000,233
Verifier is a deterministic test stub; calls model expensive work, not network timings.
```

Evaluation semantics and order choice

A false left operand of && skips its right operand; a true left operand of || skips its right operand. When the right operand is evaluated, the left side is sequenced before it. Logical operators produce 0 or 1. Bitwise & and | evaluate both operands and combine integer bits; they are not interchangeable with logical operators on arbitrary nonzero values (2&1 is zero although both values are logically true). In the experiment every condition returns 0 or 1, so the bitwise version grants the same 233 concessions but makes 1000 slow calls. It does not promise a left-to-right order for its operands. The fixture has only one counter-mutating verifier call in each expression.

For A&&B, a cheap, often-false A can avoid B; for A||B, a cheap, often-true A can avoid B. With pure independent checks, approximate expected work is cost(A)+P(A true)*cost(B) for AND and cost(A)+P(A false)*cost(B) for OR. Compare the reverse order rather than assuming 'cheapest first' is always optimal. Real dependencies and conditional probabilities also matter. The boolean result is unchanged by reordering pure commutative conditions; the number/cost of calls can change. Order is immaterial to savings when every needed operand is always evaluated anyway or both candidate orders have equal expected cost. Side effects and validity guards can make reordering incorrect even if the boolean formula seems equivalent.

Changed audit requirement

Once verification writes an audit record, reordering also changes which 166 passengers are omitted from that log. If the policy is simply 'log each actual verification', the cheap branch may still run first, but the log must be described as a verification log, not a complete passenger audit. If the new requirement is to verify and record every passenger, perform verify(i) once unconditionally, then evaluate concession using its cached result. The audit_all experiment makes exactly 1000 calls while preserving the same 233 decisions. The recommendation therefore depends on the audit completeness requirement; performance must not silently decide that policy.

Alternatives and complexity

The nested-if version implements the same lazy decision:

```
if senior:
    if eligible: grant
    else: grant only if verify_card()
else:
    grant only if verify_card()
```

For n passengers and slow-call cost C, cheap-first/nested-if work is O(n + m*C), where m is the number not already granted by the cheap branch; here m=834. Card-first, unconditional audit and bitwise versions each make n card calls, O(n+n*C). All are O(n) if C is treated as constant, but measured operation counts expose the important constant difference. Calculation state is O(1) when streaming. Persisting one audit record per verification requires O(m) storage, or O(n) for a complete audit, outside the counter-only stub.

Q3 - Device-status register (6 marks)

Representation and the completed program

Bit d is one device, with 0=OFF and 1=ON. The exercise uses unsigned char and device indices 0..7. The separate tests require CHAR_BIT==8, making the one-byte/eight-device comparison explicit. External device indices must be validated before calling these low-level functions; negative or excessive shift counts are not supported.

Operation	Bitwise expression
Set device d	reg OR (1u shifted left d)
Clear device d	reg AND complement of (1u shifted left d)
Toggle device d	reg XOR (1u shifted left d)
Test device d	(reg shifted right d) AND 1u

The complete_code.c file preserves the supplied program and fills only its four TODO bodies. Converting the results to unsigned char keeps the intended register width. 1u makes the mask unsigned.

Main statement	Predicted decimal	Predicted binary	Observed consequence
Initialize reg	0	00000000	Initial value
Switch LIGHT on	1	00000001	Bit 0 set
Switch AC on	5	00000101	First printed register
Toggle FAN	7	00000111	Bit 1 set
Switch LIGHT off	6	00000110	Second printed register
Test AC / PUMP	1 / 0	Bit 2 / bit 4	ON / OFF

show() and is_on() receive the register by value and do not alter it.

```
00000101
00000110
AC is ON, PUMP is OFF
```

Binary display, counting and signedness

show prints bits 7 down to 0 using right shift and &1u. The separate count_on function examines successive low bits. Its bit_add helper uses XOR for sum bits and (a&b)<<1 for carries, so even the counting arithmetic uses bitwise operators; loops and comparisons provide control. The count is at most eight. Exhaustive tests compare this count against an eight-element state array for every register value 0..255.

Shift has higher precedence than bitwise AND: reg & 1u << dev means reg & (1u<<dev). The alternative (reg&1u)<<dev first discards every bit except bit 0, then moves that single bit. With reg=128 and dev=7, the first result is 128 and the second is 0.

The signed-char conversion of 128 is implementation-defined when it cannot be represented. On this platform it produces -128. A signed register may therefore appear negative; promoting it to int can preserve that negative value, and right shift of a negative signed value is implementation-defined in the selected C17 rules. Left-shifting a negative signed operand is undefined. Setting bit 7 in an unsigned register itself is well-defined. Use unsigned storage and unsigned masks instead of depending on these signed effects.

```
set bit7: 134
clear AC: 130
toggle bit7: 2, on=1
reg & 1u << dev = 128; (reg & 1u) << dev = 0
This platform: signed char after converting 128 = -128
All 256 register values verified; million homes: packed=1000000 bytes, array=8000000 bytes
Function precondition: device index 0..7; validate external indices before shifting.
```

Array alternative and complexity

With char state[8], setting, clearing, toggling and testing can use state[d]=1, state[d]=0, state[d]^=1 and state[d]. For fixed valid d these operations are O(1) in both designs. One million homes use 1,000,000 versus 8,000,000 bytes for the state payload alone on this eight-bit-byte system, excluding unrelated structure padding and metadata.

For d variable devices, array scanning costs O(d), and bit scanning also costs O(d), with storage O(d) bytes versus O(ceil(d/8)) bytes. With d=8, both count routines are bounded O(1). The strict bitwise carry-adder costs up to O(log d) for an increment in a generalized implementation, so this particular bit-only counter has an O(d log d) upper bound; for eight devices this is still constant. A usual counter with ++count reduces the generalized scan bound to O(d). Arrays are easier to inspect and extend with richer state; packed bits are preferable when memory matters and a boolean state is sufficient.

Q4 - Grade and scholarship classifier (6 marks)

Rules, validation and two implementations

Grades are S for 90..100, A for 80..89, B for 70..79, C for 60..69, D for 50..59 and F for 0..49. Outside 0..100 is '?'. Every implementation checks this domain before selecting a branch or indexing a table. The ladder tests descending lower bounds. The switch uses marks/10, combining cases 10 and 9 for S; the completed supplied code returns directly from each case.

The required inputs 100,90,89,0,-1,101 produce S,S,A,F,?,? in all three correct implementations. Without the validation, 101/10=10 could incorrectly give S, while -1/10=0 under truncation toward zero could give F. Omitting case 10 would mishandle 100. The test set uses b-1,b,b+1 at every boundary 50,60,70,80,90 plus domain edges -1,0,1,99,100,101. All 105 integer inputs from -2 through 102 are also compared across ladder, switch and table.

```
100:S 95:S 83:A 50:D 49:F -5:?
```

Deliberate fall-through experiment

In a separate copy, only the S return is removed. Cases 10 and 9 then fall into case 8, so 100 and 95 become A. The other supplied outputs stay unchanged:

```
100:A 95:A 83:A 50:D 49:F -5:?
```

A switch using assignments requires break unless fall-through is intentional. A direct return also exits the function, so the correct supplied exercise needs no additional break. Standard C case labels require integer constant expressions; case 75..84 is not standard C range syntax. marks/10 works for these decade-aligned ranges, but it cannot distinguish an arbitrary 75..84 band, which spans parts of two buckets. A ladder with explicit comparisons is easier to adapt to irregular boundaries. Compiler-specific case-range extensions are avoided.

Scholarship decision and explicit missing-policy assumption

The brief does not supply income bands or scholarship categories. To demonstrate the structure, the following illustrative policy is used, not presented as the university's actual policy: income <=Rs250000 with grade S/A gives full scholarship; grade B in that band gives partial; income above Rs250000 and <=Rs500000 with grade S/A gives partial; all other valid cases give none. Negative income or invalid marks are rejected.

Nested if is chosen because there are two ordered income bands and simple grade membership checks. It expresses the band boundaries directly and handles invalid input before policy selection. Tests cover incomes 250000,250001,500000,500001 and grades S,A,B,C. A switch inside an if is reasonable if each grade gets a separate rule. A lookup table is useful for a larger stable grade-by-band policy, but then the mapping from incomes to bands still requires validation/comparisons.

Lookup alternative and complexity

```
static const char grades[11] = {'F','F','F','F','F','D','C','B','A','S','S'};
if (marks < 0 || marks > 100) return '?';
return grades[marks / 10];
```

The table uses 11 char bytes and O(1) indexing after validation. A generalized ladder has O(k) worst-case comparisons for k grade bands and O(1) extra working state, with O(k) source branches. With six fixed grades all methods are bounded constant time. A compiler may implement a switch with a jump table, comparison tree or linear tests, so its exact machine cost cannot be assumed from source syntax alone. This solution claims semantic agreement and

boundary coverage, not a measured speed superiority.

```
marks,ladder,switch,lookup
-1,?,?,?
0,F,F,F
1,F,F,F
49,F,F,F
50,D,D,D
51,D,D,D
59,D,D,D
60,C,C,C
61,C,C,C
69,C,C,C
70,B,B,B
71,B,B,B
79,B,B,B
80,A,A,A
81,A,A,A
89,A,A,A
90,S,S,S
91,S,S,S
99,S,S,S
100,S,S,S
101,?,?,?
marks,income,scholarship(0=none;1=partial;2=full)
95,250000,2
95,250001,1
95,500000,1
95,500001,0
85,250000,2
85,250001,1
85,500000,1
85,500001,0
75,250000,1
75,250001,0
75,500000,0
75,500001,0
65,250000,0
65,250001,0
65,500000,0
65,500001,0
105 marks inputs and scholarship boundaries verified.
```

Q5 - Count the work done by a loop (7 marks)

Derive the stopping condition

If n=a*b and both factors were greater than sqrt(n), their product would exceed n. Therefore a composite n has a factor no greater than sqrt(n), including equality for perfect squares. Starting at divisor 2, test only while i*i<=n. For large unsigned long values i*i can wrap and corrupt the condition; signed multiplication could instead overflow with undefined behaviour. Since i is positive, i<=n/i expresses the same bound without multiplying. n<2 is rejected before the loop. Return immediately when a remainder is zero.

The completed-code exercise fills only the TODO region. It uses the equivalent overflow-safe guard i<=n/i. As in the expected output, checks counts the divisibility operations n%i; it does not count the division introduced by the safe guard. loops.c reports those guard divisions separately, so the displayed expected counts are not misrepresented as every division expression in the program. A compiler can combine quotient/remainder calculations, so source operation counts also do not claim hardware instruction counts.

Predicted and observed work

Ticket	Prime?	Full trial prediction / actual	Improved prediction / actual
97	Yes	95 / 95	8 / 8
100	No	1 / 1	1 / 1
7919	Yes	7917 / 7917	87 / 87
1000003	Yes	1000001 / 1000001	999 / 999

For prime 97 the candidates are 2..9: eight checks. For 7919 they are 2..88: 87. For 1000003 they are 2..1000: 999. Ticket 100 stops at divisor 2. Changing only the guard to i<n produces the full-trial counts above. The complete exercise prints:

```
97  prime      divisions = 8
     100  not prime  divisions = 1
    7919  prime      divisions = 87
 1000003  prime      divisions = 999
```

for, while and do-while

for and while check the bound before their first body execution. The do-while body runs at least once, so n=2 and n=3 must return true first; otherwise testing divisor 2 would incorrectly reject 2. For n>=4, divisor 2 is always within the square-root bound and is safe as the first body execution. All three improved versions perform the same remainder tests; do-while saves its initial guard division after those small-value checks.

```
n,prime,full_remainders,for_remainders,while_remainders,do_remainders,for_guard_divisions,do_guard_
    divisions
0,0,0,0,0,0,0,0
1,0,0,0,0,0,0,0
2,1,0,0,0,0,1,0
3,1,1,0,0,0,1,0
4,0,1,1,1,1,1,0
9,0,2,2,2,2,2,1
97,1,95,8,8,8,9,8
100,0,1,1,1,1,1,0
7919,1,7917,87,87,87,88,87
1000003,1,1000001,999,999,999,1000,999
All four versions agree for 0..10000.
```

The output includes 0,1,2,3,4 and 9 to expose non-primes below two, small primes and perfect squares. All four variants agree on 0..10000. The exhaustive redesign check in Q7 covers all values through the maximum ticket. A return is used for early composite detection; break would require a flag and a correct post-loop result. A continue belongs only where the divisor progression is guaranteed, otherwise it can accidentally skip an increment in a while loop. The sieve in Q7 uses continue to skip composite base candidates.

Complexity and sieve alternative

Full trial uses O(n) remainder tests in the prime worst case and O(1) working space. Improved trial uses O(sqrt(n)) tests, with O(1) state; both can return after one test for an even composite. The count here treats fixed-width arithmetic as O(1). Arbitrary-precision arithmetic would require a separate bit-complexity model.

For many bounded queries, sieve an array once:

```
allocate flags[0..N], initially true
flags[0] = flags[1] = false
for p = 2 while p <= N/p:
    if flags[p] is false: continue
    for multiple = p*p to N in steps of p:
        flags[multiple] = false
answer each ticket n with flags[n], after checking 0 <= n <= N
```

Starting at p*p avoids redoing multiples already covered by smaller primes. Preprocessing takes O(N log log N), queries O(1), and storage O(N). The inclusive 0..N indexing needs N+1 flags, not N flags. For a few tickets or very large N, trial

division avoids setup and a large allocation. Q7 implements and measures this alternative for the bounded repeated-query case.

Q6 - Loan repayment and termination (8 marks)

Monetary model and loop choice

All money is long long paise. A rate of 200 basis points means 2% per month. Interest is added first, rounded half-up once per month using (balance*rate_bp+5000)/10000, followed by payment. The brief does not define a rounding policy; this policy is explicit and used consistently in the hand calculation and program. A while loop fits an unknown month count and skips the schedule entirely if the initial balance is zero.

```
validate principal, monthly rate, repayment and month limit
while balance > 0:
    if months == safety_limit: status=LIMIT_REACHED; break
    interest = round_half_up(balance * monthly_rate)
    if repayment <= interest: status=NON_AMORTIZING; break
    actual_payment = minimum(repayment, balance + interest)
    balance = balance + interest - actual_payment
    accumulate interest and actual payment; increment months
    print the month's row
```

The final payment is capped at amount due, so the balance reaches exactly zero without an overpayment. Floating balance!=0 can be unsafe because representation and subtraction may leave a tiny residual or a negative balance; epsilon tests can also obscure contractual rounding. Integer paise makes this stopping condition exact within the chosen monetary policy.

If repayment is below interest, debt increases; if equal, it does not decrease. Neither is a successful repayment plan. The program rejects such a schedule before issuing a fictitious payment row. With a fixed non-negative rate and rounding, a strictly decreasing balance cannot make monthly interest increase, so an initially amortizing payment continues reducing the debt. A reduction of at least one paisa per month gives mathematical termination, but that alone is not a practical duration guarantee.

Independent hand schedule, verified row by row

Month	Opening (Rs)	Interest (Rs)	Payment (Rs)	Closing (Rs)
1	10000.00	200.00	2000.00	8200.00
2	8200.00	164.00	2000.00	6364.00
3	6364.00	127.28	2000.00	4491.28
4	4491.28	89.83	2000.00	2581.11
5	2581.11	51.62	2000.00	632.73
6	632.73	12.65	645.38	0.00

Monthly rounded interest totals Rs645.38; total payments are Rs10645.38. Month 4 interest 89.8256 rounds to 89.83, month 5 interest 51.6222 rounds to 51.62, and month 6 interest 12.6546 rounds to 12.65. The program asserts six months, the exact interest and paid totals, and checks the six independently specified interest amounts.

Growth as repayment approaches interest

The prediction is that repayment closer to the first month's Rs200 interest requires more iterations, with no repayment at or below Rs200. The measured results confirm that trend:

Monthly repayment (Rs)	Executed months	Status	Remaining balance (Rs)
2000.00	6	PAID	0
1000.00	12	PAID	0
500.00	26	PAID	0
250.00	82	PAID	0
210.00	154	PAID	0
201.00	268	PAID	0
200.01	360	LIMIT_REACHED	9425.52
200.00	0	NON_AMORTIZING	10000.00
199.99	0	NON_AMORTIZING	10000.00

The 360 limit is a deployment policy, not evidence of repayment or proof the loan could never finish. On reaching it, report 'schedule incomplete after 360 months', the remaining balance, and request revised terms or an explicitly approved longer projection. Do not label the result PAID. The test with Rs200.01 leaves Rs9425.52 outstanding; the unrounded formula estimates 501 months, so the limit is meaningful. Zero principal, zero rate, invalid negative principal, zero payment, invalid rate and invalid limit are also tested.

Bounds and formula alternative

Accepted principal and payment are each at most 10^12 paise, rate at most 10000 basis points (100% monthly), and safety limit at most 100000 months. A non-amortizing schedule stops before accumulation; an accepted one has balance no greater than principal. Thus balance*rate+5000 is at most 10^16+5000, due at most 2*10^12, and accumulated interest/payment at most about 10^17 paise, safely below the minimum long long maximum. These are validation limits for the classroom function, not suggested loan terms.

The formula n=-log(1-r*P/E)/log(1+r) comes from a constant-rate, constant-payment recurrence without monthly rounding. For P>0, r>0, it requires E>r*P; use ceil(n), log1p(-r*P/E), and log1p(r) to reduce cancellation. Handle r=0 separately as ceil(P/E), P=0 as zero, and E<=r*P as non-amortizing. The demonstration applies the formula only after supplying valid bounded fixtures. Near E=r*P the argument approaches zero and the estimate becomes sensitive to rounding.

Formula evaluation is O(1) time/O(1) state in the fixed-precision operation model; simulation is O(m) for m months, capped by the safety limit, and O(1) working state when rows stream out. Storing all rows takes O(m). The formula cannot produce a full month-by-month schedule or enforce the selected monthly-paisa rounding by itself. It is useful as an estimate and cross-check, while integer simulation is retained for the issued schedule. Formula and rounded simulation match the paid fixtures here; that is not a guarantee for every input.

```
month,opening_paise,interest_paise,payment_paise,closing_paise
1,1000000,20000,200000,820000
2,820000,16400,200000,636400
3,636400,12728,200000,449128
4,449128,8983,200000,258111
5,258111,5162,200000,63273
6,63273,1265,64538,0
Baseline: months=6 total_interest=64538 total_paid=1064538 paise
payment_paise,months,status,remaining_paise,total_interest_paise,unrounded_formula_months
200000,6,PAID,0,64538,6
100000,12,PAID,0,127032,12
50000,26,PAID,0,289875,26
25000,82,PAID,0,1031912,82
21000,154,PAID,0,2228652,154
20100,268,PAID,0,4383119,268
20001,360,LIMIT_REACHED,942552,7142912,501
20000,0,NON_AMORTIZING,1000000,0,inf
19999,0,NON_AMORTIZING,1000000,0,inf
Hand schedule, zero-rate/principal, final payment, invalid data and safety-limit checks passed.
```

Q7 - Diagnose, redesign and validate (13 marks)

A. Diagnose the bottleneck (2 marks)

Q5's square-root trial repeats the same divisibility questions every time a ticket is checked. For 1000 queries cycling 97,100,7919,1000003, the measured workload is 273750 remainder tests plus 274500 safe-bound guard divisions; it returns 750 prime answers. The large prime alone causes 999 remainder tests on each of its 250 appearances, or 249750 tests, about 91.2% of the total. Switching for to while cannot remove this repeated mathematical work.

B. Define a measurable objective (2 marks)

For ticket numbers bounded by 1000003, remove at least 99% of repeated query remainder tests on the same 1000-query workload while preserving every primality answer. Report preprocessing operations and memory separately. A second objective is O(1) lookup per future ticket. This is an operation-count objective, not a promise that a single short batch will run faster after including setup.

C. Revised design and trade-offs (4 marks)

Build a Sieve of Eratosthenes once for the allowed ticket domain, then reuse it. The exploited property is that every composite has a prime factor no greater than its square root, so marking multiples of each prime identifies composites for all later queries. The table generalizes beyond a cache of the four repeated sample values: it supports any future ticket in range.

```
allocate N+1 byte flags; reject allocation failure
initialize flags true; set flags 0 and 1 false
for candidate p=2 while p<=N/p:
    if flags[p] is false: continue
    mark p*p, p*p+p, ... up to N as composite
for each validated ticket in 0..N:
    answer using flags[ticket]
release flags when the reusable query session ends
```

The fixed N=1000003 keeps p*p and multiple+p within unsigned long range. A general implementation accepting arbitrary N would need additional size/allocation/loop-overflow checks. This program uses only known in-range fixtures and validates all possible domain answers. It handles allocation failure and frees the table. For an externally supplied ticket, domain validation must precede indexing.

Extra memory is 1000004 bytes for unsigned-char flags on this machine, plus small loop state. A bit-packed sieve could lower storage, but would add masking code. Setup is O(N log log N), a batch of q lookups is O(q), and persistent table storage is O(N). Square-root trial is O(q sqrt(N)) worst-case with O(1) state. For sparse enormous ticket numbers the array may be unaffordable; a segmented approach or trial division may be more suitable.

D. Predict before execution (1 mark)

The recorded prediction was (8+1+87+999)*250=273750 original remainder tests versus zero query remainders for the sieve, 1000 direct lookups, 1000004 flag bytes and 750 prime answers. The prediction explicitly allowed nonzero initialization and marking work; it did not assume a free sieve build.

E. Implement and validate (2 marks)

```
method,queries,prime_answers,query_remainders,query_guard_divisions,flag_bytes
sqrt_trial,1000,750,273750,274500,0
sieve,1000,750,0,0,1000004
Preprocessing: initialized_flags=1000004 outer_tests=999 marking_writes=2122051
CPU_ms: trial=2.148 sieve_build=1.971 lookups=0.002
Exhaustive validation: all 1000004 inputs from 0 through 1000003 agree.
Measured objective: 100% fewer query remainder tests; preprocessing and memory are additional
    costs.
```

The same 1000 fixtures were evaluated by both methods. Additionally, every integer 0..1000003 was checked for agreement, including 0/1, primes, composites, perfect squares and the maximum ticket. Sanitized execution also passed. Exhaustive verification time is deliberately excluded from the displayed query/build timing; verification is a

development cost, not a required step for each deployed batch.

Measurement	Original square-root trial	Redesigned sieve
Queries	1000	1000
Prime answers	750	750
Query remainder tests	273750	0
Query guard divisions	274500	0
Direct flag lookups	0	1000
Flag storage	0	1000004 bytes
Initialization writes	0	1000004 flags, plus clearing 0 and 1
Sieve marking writes	0	2122051
Base-candidate flag tests	0	999

The sieve's safe outer loop also evaluates its n/p bound 1000 times, including its final failed guard. Marking writes include repeated writes to an already-composite flag and are counted honestly. These operations have different costs; zero query remainders does not mean zero total work.

F. Defend the result (2 marks)

The query-remainder reduction is 100%, exceeding the 99% objective. All 1000004 validation inputs agree, and the answer count stays 750. The memory/setup cost is the main new trade-off. Three executions produced:

Run	Trial CPU ms	Sieve build CPU ms	Lookup CPU ms
1	2.148	1.971	0.002
2	0.249	0.709	0.001
3	0.247	0.617	0.000

The median trial time is 0.249 ms, median build time 0.709 ms and median lookup time 0.001 ms. These short CPU-clock samples vary with warm-up, optimization and scheduling; a displayed 0.000 is below reported resolution, not zero elapsed time. On the median measurements, setup plus one batch is slower than trial division. Reusing an already-built table makes later queries cheap, and the measured operation-count objective is still achieved. More sustained workloads would be needed for a robust runtime crossover claim.

Retain trial division for occasional or unbounded queries. Use a reusable sieve for dense or repeated queries within a manageable known maximum, where its table can be amortized across many batches. This is a change in information reuse and algorithmic work, rather than a cosmetic replacement of a loop statement.

References

The supplied CSA02 Assignment 2 PDF defines Q1-Q7 and their marks. C language rules were checked against WG14 N1570 (a publicly available C11 committee draft), sections 6.5, 6.5.7, 6.5.13-16 and 6.8.4-6; these operator/statement rules remain applicable to the selected C17 examples: https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf. GCC's Warning Options manual documents -Wall and -Wsequence-point: https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html. The actual diagnostic is from Apple Clang, as recorded with Q1; the reference is not a claim that GNU GCC was executed.

Appendix - C source listings

Each listing is labelled with its folder and filename. Related files are included in the companion ZIP and repository. Deliberate error examples are identified in their task explanations. Long lines are visually wrapped; use the original source files for execution.

Task01/billing.c

```
#include <stdio.h>
#include <assert.h>
/* Monetary domain: non-negative paise, these bounded classroom fixtures. */
static long long percent(long long amount, unsigned rate) {
    return (amount * rate + 50) / 100; /* half-up at one paisa */
}
int main(void) {
    const int price[] = {100,999,199}, qty[] = {10,1,3}, senior[] = {1,0,1};
    const int predicted_total[] = {900,1048,537}, predicted_net[] = {900,998,537};
    const long long predicted_correct[] = {89250,99895,53282};

        puts("price,qty,senior,predicted_total,observed_total,predicted_net,observed_net,correct_pa
        ise");
    for (int i=0;i<3;i++) {
        int total=price[i]*qty[i]-price[i]*qty[i]*15/100*senior[i]+price[i]*qty[i]*5/100;
        int net=total>1000?total-50:total;
        long long gross=(long long)price[i]*100*qty[i];
        long long discount=senior[i]?percent(gross,15):0;
        long long taxable=gross-discount, tax=percent(taxable,5);
        long long after_tax=taxable+tax;
        long long payable=after_tax>100000?after_tax-5000:after_tax;
        assert(total==predicted_total[i] && net==predicted_net[i]);
        assert(payable==predicted_correct[i]);
        printf("%d,%d,%d,%d,%d,%d,%d,%lld\n",price[i],qty[i],senior[i],
               predicted_total[i],total,predicted_net[i],net,payable);
        printf("gross=%lld discount=%lld taxable=%lld GST=%lld payable=%lld paise\n",
               gross,discount,taxable,tax,payable);
    }
    int count=0; ++count; assert(count==1);
    puts("Correct counter update: ++count; count=1");
    assert((100000>100000?100000-5000:100000)==100000);
    assert((100001>100000?100001-5000:100001)==95001);
    puts("Rebate threshold: 100000 paise unchanged; 100001 paise -> 95001");
    return 0;
}
```

Task01/undefined_example.c

```
/* Compile for diagnostics only. Do not execute undefined behaviour. */
int main(void) { int count=0; count=count++ + 1; return count; }
```

Task02/short_circuit.c

```
#include <stdio.h>
#include <assert.h>
static unsigned calls;
static int verify(unsigned i) { ++calls; return i%10==0; }
static int senior(unsigned i) { return i%4==0; }
static int eligible(unsigned i) { return i%3!=0; }
int main(void) {
    calls=0; int answer=0 && verify(0); assert(!answer && calls==0);
    printf("0 && verify: calls=%u\n",calls);
    calls=0; answer=1 || verify(0); assert(answer && calls==0);
    printf("1 || verify: calls=%u\n",calls);
    calls=0; answer=0 & verify(0); assert(!answer && calls==1);
    printf("0 & verify: calls=%u\n",calls);
    const char *names[]={"eligible_first","card_first","nested_if","bitwise","audit_all"};
    unsigned expected[]={834,1000,834,1000,1000};
    puts("version,passengers,card_calls,grants");
    for (unsigned version=0;version<5;version++) {
        calls=0;unsigned grants=0;
        for (unsigned i=0;i<1000;i++) {
            int grant=0;
            switch(version) {
            case 0:grant=(senior(i)&&eligible(i))||verify(i);break;
            case 1:grant=verify(i)||(senior(i)&&eligible(i));break;
            case 2:
                if (senior(i)) {
                    if (eligible(i)) grant=1;
                    else grant=verify(i);
                } else grant=verify(i);
                break;
            case 3:grant=(senior(i)&eligible(i))|verify(i);break;
            default: {
                int audited_card=verify(i); /* independent mandatory audit step */
                grant=(senior(i)&&eligible(i))||audited_card;break;
            }
            }
            grants+=(unsigned)grant;
        }
        assert(calls==expected[version] && grants==233);
        printf("%s,1000,%u,%u\n",names[version],calls,grants);
    }
    puts("Verifier is a deterministic test stub; calls model expensive work, not network
        timings.");
    return 0;
}
```

Task03/complete_code.c

```c
#include <stdio.h>
/* Each bit of one byte stores the ON/OFF state of one device. */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };
unsigned char switch_on(unsigned char reg, int dev)
{
    // TODO 1: Set the bit of device dev to 1 and return the new register value
    return (unsigned char)(reg | (1u << dev));
}
unsigned char switch_off(unsigned char reg, int dev)
{
    // TODO 2: Clear the bit of device dev to 0 and return the new register value
    return (unsigned char)(reg & ~(1u << dev));
}
unsigned char toggle(unsigned char reg, int dev)
{
    // TODO 3: Flip the bit of device dev and return the new register value
    return (unsigned char)(reg ^ (1u << dev));
}
int is_on(unsigned char reg, int dev)
{
    // TODO 4: Return 1 if the bit of device dev is 1, otherwise 0
    return (reg >> dev) & 1u;
}
void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}
int main(void)
{
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");
    return 0;
}
```

Task03/register_tests.c

```
#include <stdio.h>
#include <assert.h>
#include <limits.h>
_Static_assert(CHAR_BIT==8,"Eight-bit byte required for this memory comparison");
/* Bitwise ripple-carry addition keeps the counting arithmetic bitwise too. */
static unsigned bit_add(unsigned a,unsigned b) {
    while (b) { unsigned carry=(a&b)<<1; a^=b; b=carry; }
    return a;
}
static unsigned count_on(unsigned char reg) {
    unsigned count=0;
    while(reg) { count=bit_add(count,reg&1u);reg=(unsigned char)(reg>>1); }
    return count;
}
int main(void) {
    unsigned char reg=6;
    reg=(unsigned char)(reg|(1u<<7)); assert(reg==134);
    printf("set bit7: %u\n",(unsigned)reg);
    reg=(unsigned char)(reg&~(1u<<2)); assert(reg==130);
    printf("clear AC: %u\n",(unsigned)reg);
    reg=(unsigned char)(reg^(1u<<7)); assert(reg==2);
    printf("toggle bit7: %u, on=%u\n",(unsigned)reg,count_on(reg));
    reg=128;int dev=7;
    printf("reg & 1u << dev = %u; (reg & 1u) << dev = %u\n",reg&1u<<dev,(reg&1u)<<dev);
    signed char signed_reg=(signed char)reg; /* implementation-defined if not representable */
    printf("This platform: signed char after converting 128 = %d\n",signed_reg);
    for(unsigned value=0;value<256;value++) {
        unsigned array_count=0;char state[8];
        for(unsigned bit=0;bit<8;bit++) {
            state[bit]=(char)((value>>bit)&1u);array_count+=(unsigned)state[bit]; }
        assert(count_on((unsigned char)value)==array_count);
    }
    printf("All 256 register values verified; million homes: packed=%zu bytes, array=%zu bytes\n",
           1000000*sizeof(unsigned char),1000000*sizeof(char[8]));
    puts("Function precondition: device index 0..7; validate external indices before shifting.");
    return 0;
}
```

Task04/classifier.c

```
#include <stdio.h>
#include <assert.h>
static char ladder(int m) {
    if(m<0||m>100)return '?';
    if(m>=90)return 'S';else if(m>=80)return 'A';else if(m>=70)return 'B';
    else if(m>=60)return 'C';else if(m>=50)return 'D';else return 'F';
}
static char by_switch(int m) {
    if(m<0||m>100)return '?';
    switch(m/10){case 10:case 9:return 'S';case 8:return 'A';case 7:return 'B';
    case 6:return 'C';case 5:return 'D';default:return 'F';}
}
static char lookup(int m) {
    static const char grades[11]={'F','F','F','F','F','D','C','B','A','S','S'};
    return (m<0||m>100)?'?':grades[m/10];
}
/* Illustrative rules, because the assignment supplies no scholarship thresholds.
   Income <=250000: S/A full, B partial. Income <=500000: S/A partial.
   Others none. Returns -1 invalid, 0 none, 1 partial, 2 full. */
static int scholarship(int marks,long income) {
    char g=ladder(marks);
    if(g=='?'||income<0)return -1;
    if(income<=250000) {
        if(g=='S'||g=='A')return 2;
        if(g=='B')return 1;
    } else if(income<=500000 && (g=='S'||g=='A'))return 1;
    return 0;
}
int main(void) {
    int inputs[]={-1,0,1,49,50,51,59,60,61,69,70,71,79,80,81,89,90,91,99,100,101};
    puts("marks,ladder,switch,lookup");
    for(unsigned i=0;i<sizeof inputs/sizeof inputs[0];i++) {
        int m=inputs[i];char g=ladder(m);
        assert(g==by_switch(m)&&g==lookup(m));
        printf("%d,%c,%c,%c\n",m,g,by_switch(m),lookup(m));
    }
    for(int m=-2;m<=102;m++)assert(ladder(m)==by_switch(m)&&ladder(m)==lookup(m));
    int marks[]={95,85,75,65};long incomes[]={250000,250001,500000,500001};
    puts("marks,income,scholarship(0=none;1=partial;2=full)");
    for(unsigned i=0;i<4;i++)for(unsigned j=0;j<4;j++)
        printf("%d,%ld,%d\n",marks[i],incomes[j],scholarship(marks[i],incomes[j]));
    assert(scholarship(90,250000)==2 && scholarship(90,250001)==1);
    assert(scholarship(90,500000)==1 && scholarship(90,500001)==0);
    assert(scholarship(70,250000)==1 && scholarship(70,250001)==0);
    assert(scholarship(101,100)==-1 && scholarship(90,-1)==-1);
    puts("105 marks inputs and scholarship boundaries verified.");return 0;
}
```

Task04/complete_code.c

```
#include <stdio.h>
/* Grade rules: 90-100 S, 80-89 A, 70-79 B, 60-69 C, 50-59 D, below 50 F, invalid '?' */
char grade(int marks)
{
    if (marks < 0 || marks > 100) return '?';
    // TODO 1: Use a switch on marks / 10 to return the grade letter (combine cases
    //   where possible)
    switch (marks / 10) {
        case 10: case 9: return 'S';
        case 8: return 'A';
        case 7: return 'B';
        case 6: return 'C';
        case 5: return 'D';
        default: return 'F';
    }
}
int main(void)
{
    int marks[] = {100, 95, 83, 50, 49, -5};
    for (int i = 0; i < 6; i++) printf("%d:%c ", marks[i], grade(marks[i]));
    printf("\n");
    return 0;
}
```

Task04/fallthrough.c

```
#include <stdio.h>
/* Grade rules: 90-100 S, 80-89 A, 70-79 B, 60-69 C, 50-59 D, below 50 F, invalid '?' */
char grade(int marks)
{
    if (marks < 0 || marks > 100) return '?';
    // TODO 1: Use a switch on marks / 10 to return the grade letter (combine cases
    //   where possible)
    switch (marks / 10) {
        case 10: case 9: /* S return deliberately removed: falls through */
        case 8: return 'A';
        case 7: return 'B';
        case 6: return 'C';
        case 5: return 'D';
        default: return 'F';
    }
}
int main(void)
{
    int marks[] = {100, 95, 83, 50, 49, -5};
    for (int i = 0; i < 6; i++) printf("%d:%c ", marks[i], grade(marks[i]));
    printf("\n");
    return 0;
}
```

Task05/complete_code.c

```
#include <stdio.h>
unsigned long checks = 0;     /* number of divisions performed by the last call */
/* Returns 1 if n is prime, otherwise 0. */
int is_prime(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;
    // TODO 1: Try divisors i = 2, 3, 4 ... only while i * i <= n, adding 1 to checks
    //   for every division; return 0 as soon as a divisor is found, otherwise return 1
    for (unsigned long i=2; i<=n/i; ++i) {
        ++checks;
        if(n%i==0)return 0;
    }
    return 1;
}
int main(void)
{
    unsigned long tickets[] = {97, 100, 7919, 1000003};
    for (int k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%8lu  %-9s  divisions = %lu\n", tickets[k],
               p ? "prime" : "not prime", checks);
    }
    return 0;
}
```

Task05/full_trial.c

```
#include <stdio.h>
unsigned long checks = 0;     /* number of divisions performed by the last call */
/* Returns 1 if n is prime, otherwise 0. */
int is_prime(unsigned long n)
{
    checks = 0;
    if (n < 2) return 0;
    // TODO 1: Try divisors i = 2, 3, 4 ... only while i * i <= n, adding 1 to checks
    //   for every division; return 0 as soon as a divisor is found, otherwise return 1
    for (unsigned long i=2; i<n; ++i) {
        ++checks;
        if(n%i==0)return 0;
    }
    return 1;
}
int main(void)
{
    unsigned long tickets[] = {97, 100, 7919, 1000003};
    for (int k = 0; k < 4; k++) {
        int p = is_prime(tickets[k]);
        printf("%8lu  %-9s  divisions = %lu\n", tickets[k],
               p ? "prime" : "not prime", checks);
    }
    return 0;
}
```

Task05/loops.c

```
#include <stdio.h>
#include <assert.h>
#include "prime.h"
int main(void) {
    unsigned long numbers[]={0,1,2,3,4,9,97,100,7919,1000003};
    puts("n,prime,full_remainders,for_remainders,while_remainders,do_remainders,for_guard_divisions
        ,do_guard_divisions");
    for(unsigned k=0;k<sizeof numbers/sizeof numbers[0];k++) {
        unsigned long n=numbers[k];Counts a,b,c,d;
        bool p=prime_full(n,&a),q=prime_for(n,&b),r=prime_while(n,&c),s=prime_do(n,&d);
        assert(p==q&&q==r&&r==s&&b.remainders==c.remainders&&c.remainders==d.remainders);
        printf("%lu,%d,%lu,%lu,%lu,%lu,%lu,%lu\n",n,p,a.remainders,b.remainders,c.remainders,d.rema
            inders,b.guards,d.guards);
    }
    for(unsigned long n=0;n<=10000;n++) {
        Counts a,b,c,d;bool p=prime_full(n,&a);
        assert(p==prime_for(n,&b)&&p==prime_while(n,&c)&&p==prime_do(n,&d));
    }
    puts("All four versions agree for 0..10000.");return 0;
}
```

Task05/prime.c

```
#include "prime.h"
bool prime_full(unsigned long n,Counts *c) {
    *c=(Counts){0,0};if(n<2)return false;
    for(unsigned long i=2;i<n;i++){++c->remainders;if(n%i==0)return false;}
    return true;
}
static bool within(unsigned long i,unsigned long n,Counts *c) {
    ++c->guards;return i<=n/i; /* no i*i overflow; also performs a division */
}
bool prime_for(unsigned long n,Counts *c) {
    *c=(Counts){0,0};if(n<2)return false;
    for(unsigned long i=2;within(i,n,c);i++){++c->remainders;if(n%i==0)return false;}
    return true;
}
bool prime_while(unsigned long n,Counts *c) {
    *c=(Counts){0,0};if(n<2)return false;unsigned long i=2;
    while(within(i,n,c)){++c->remainders;if(n%i==0)return false;++i;}
    return true;
}
bool prime_do(unsigned long n,Counts *c) {
    *c=(Counts){0,0};if(n<2)return false;if(n<4)return true;
    unsigned long i=2;
    do {++c->remainders;if(n%i==0)return false;++i;}while(within(i,n,c));
    return true;
}
```

Task05/prime.h

```
#ifndef CSA02_PRIME_H
#define CSA02_PRIME_H
#include <stdbool.h>
typedef struct { unsigned long remainders, guards; } Counts;
bool prime_full(unsigned long n,Counts *c);
bool prime_for(unsigned long n,Counts *c);
bool prime_while(unsigned long n,Counts *c);
bool prime_do(unsigned long n,Counts *c);
#endif
```

Task06/loan.c

```
#include "loan.h"
#include <stdio.h>
/* Domain bounds make balance*rate_bp+5000 and totals safe in long long.
   rate_bp=200 means 2% per month. Monthly interest rounded half-up to paise. */
Loan simulate(long long principal,unsigned rate_bp,long long payment,unsigned limit,bool
    print_rows) {
    Loan r={INVALID,0,0,0,principal};
    if(principal<0||principal>1000000000000LL||rate_bp>10000||payment<=0||
       payment>1000000000000LL||limit==0||limit>100000)return r;
    r.status=PAID;
    if(print_rows)puts("month,opening_paise,interest_paise,payment_paise,closing_paise");
    while(r.balance>0) {
        if(r.months==limit){r.status=LIMIT_REACHED;break;}
        long long interest=(r.balance*rate_bp+5000)/10000;
        if(payment<=interest){r.status=NON_AMORTIZING;break;}
        long long due=r.balance+interest,actual=due<payment?due:payment;
        long long opening=r.balance;
        r.balance=due-actual;r.interest+=interest;r.paid+=actual;++r.months;

            if(print_rows)printf("%u,%lld,%lld,%lld,%lld\n",r.months,opening,interest,actual,r.bala
            nce);
    }
    return r;
}
```

Task06/loan.h

```
#ifndef CSA02_LOAN_H
#define CSA02_LOAN_H
#include <stdbool.h>
typedef enum { PAID, NON_AMORTIZING, LIMIT_REACHED, INVALID } Status;
typedef struct { Status status; unsigned months; long long interest,paid,balance; } Loan;
Loan simulate(long long principal,unsigned rate_bp,long long payment,unsigned limit,bool
    print_rows);
#endif
```

Task06/main.c

```
#include <stdio.h>
#include <assert.h>
#include <math.h>
#include "loan.h"
static const char *name(Status s) {
    switch(s){case PAID:return "PAID";case NON_AMORTIZING:return "NON_AMORTIZING";
    case LIMIT_REACHED:return "LIMIT_REACHED";default:return "INVALID";}
}
static double formula(double principal,double rate,double payment) {
    if(principal==0)return 0;
    if(rate==0)return ceil(principal/payment);
    if(payment<=rate*principal)return INFINITY;
    return ceil(-log1p(-rate*principal/payment)/log1p(rate));
}
int main(void) {
    Loan base=simulate(1000000,200,200000,360,true);
    assert(base.status==PAID&&base.months==6&&base.interest==64538&&base.paid==1064538&&base.balanc
        e==0);
    printf("Baseline: months=%u total_interest=%lld total_paid=%lld
        paise\n",base.months,base.interest,base.paid);
    long long amounts[]={200000,100000,50000,25000,21000,20100,20001,20000,19999};

        puts("payment_paise,months,status,remaining_paise,total_interest_paise,unrounded_formula_mo
        nths");
    for(unsigned i=0;i<sizeof amounts/sizeof amounts[0];i++){
        Loan r=simulate(1000000,200,amounts[i],360,false);

            printf("%lld,%u,%s,%lld,%lld,%.0f\n",amounts[i],r.months,name(r.status),r.balance,r.int
            erest,
               formula(1000000,0.02,(double)amounts[i]));
        assert(r.months<=360);
        if(r.status==PAID)assert(r.paid==1000000+r.interest);
    }
    assert(simulate(10000,0,3000,360,false).months==4);
    assert(simulate(0,200,2000,360,false).months==0);
    assert(simulate(1000000,200,20000,360,false).status==NON_AMORTIZING);
    assert(simulate(1000000,200,20001,360,false).status==LIMIT_REACHED);
    assert(simulate(1000000,200,200000,2,false).status==LIMIT_REACHED);
    assert(simulate(-1,200,100,360,false).status==INVALID);
    assert(simulate(10000,200,0,360,false).status==INVALID);
    assert(simulate(10000,10001,100,360,false).status==INVALID);
    assert(simulate(10000,200,100,0,false).status==INVALID);
    /* Independently stated hand calculation, including each rounded interest. */
    const long long openings[]={1000000,820000,636400,449128,258111,63273};
    const long long expected_interest[]={20000,16400,12728,8983,5162,1265};
    for(unsigned i=0;i<6;i++)assert((openings[i]*200+5000)/10000==expected_interest[i]);
    puts("Hand schedule, zero-rate/principal, final payment, invalid data and safety-limit checks
        passed.");
    return 0;
}
```

Task07/redesign.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>
#include <time.h>
#include "../Task05/prime.h"
int main(void) {
    const unsigned long N=1000003, tickets[]={97,100,7919,1000003};
    unsigned char *flags=malloc((size_t)N+1);
    if(!flags){fputs("Cannot allocate sieve; no results issued\n",stderr);return 2;}
    clock_t start=clock();
    memset(flags,1,(size_t)N+1);flags[0]=flags[1]=0;
    unsigned long long marks=0,outer_tests=0;
    for(unsigned long p=2;p<=N/p;p++){
        ++outer_tests;
        if(!flags[p])continue; /* composites already have a smaller prime factor */
        for(unsigned long multiple=p*p;multiple<=N;multiple+=p){flags[multiple]=0;++marks;}
    }
    double build_ms=1000.0*(clock()-start)/CLOCKS_PER_SEC;
    unsigned long long rem=0,guards=0;unsigned original_primes=0,new_primes=0;
    start=clock();
    for(unsigned i=0;i<1000;i++){
        Counts c;original_primes+=prime_for(tickets[i%4],&c);rem+=c.remainders;guards+=c.guards;
    }
    double trial_ms=1000.0*(clock()-start)/CLOCKS_PER_SEC;
    start=clock();
    for(unsigned i=0;i<1000;i++)new_primes+=flags[tickets[i%4]];
    double lookup_ms=1000.0*(clock()-start)/CLOCKS_PER_SEC;
    assert(rem==273750 && original_primes==750 && new_primes==original_primes);
    puts("method,queries,prime_answers,query_remainders,query_guard_divisions,flag_bytes");
    printf("sqrt_trial,1000,%u,%llu,%llu,0\n",original_primes,rem,guards);
    printf("sieve,1000,%u,0,0,%zu\n",new_primes,(size_t)N+1);
    printf("Preprocessing: initialized_flags=%zu outer_tests=%llu
        marking_writes=%llu\n",(size_t)N+1,outer_tests,marks);
    printf("CPU_ms: trial=%.3f sieve_build=%.3f lookups=%.3f\n",trial_ms,build_ms,lookup_ms);
    for(unsigned long n=0;n<=N;n++){Counts c;assert((flags[n]!=0)==prime_for(n,&c));}
    printf("Exhaustive validation: all %lu inputs from 0 through %lu agree.\n",N+1,N);
    puts("Measured objective: 100% fewer query remainder tests; preprocessing and memory are
        additional costs.");
    free(flags);return 0;
}
```